SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A **Dry**

Run Evaluation

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph: A

/ \

B C

/\ /\

D E F G

Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration Queue Visited Node

1
2
3
4
5
6
7

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,

A

/ \

B C

/\ /\

D E F G

Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion
1
2
3
4
5

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.

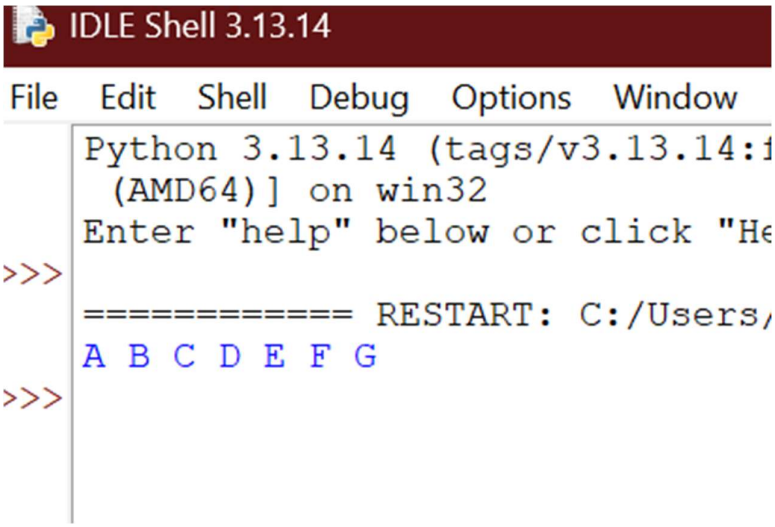
			procedural errors.		
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

1. Breadth First Search (BFS)

```
graph = {
   'A': ['B', 'C'],
   'B': ['D', 'E'],
   'C': ['F', 'G'],
   'D': [],
   'E': [],
   'F': [],
   'G': []
}
visited = []
queue = []
start = 'A'
visited.append(start)
queue.append(start)
while queue:
   node = queue.pop(0)
```

```
    print(node, end=" ")
    for i in graph[node]:
        if i not in visited:
            visited.append(i)
            queue.append(i)
```

Output



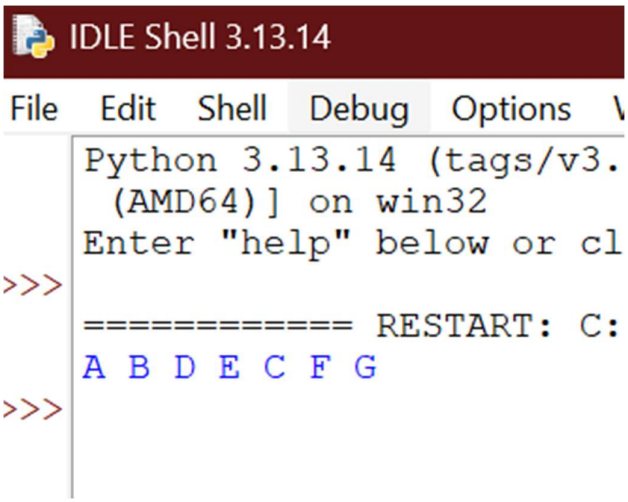
---

2. Depth First Search (DFS)

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}
visited = []
def dfs(node):
    if node not in visited:
```

```
        print(node, end=" ")
        visited.append(node)
        for i in graph[node]:
            dfs(i)
dfs('A')
```

Output



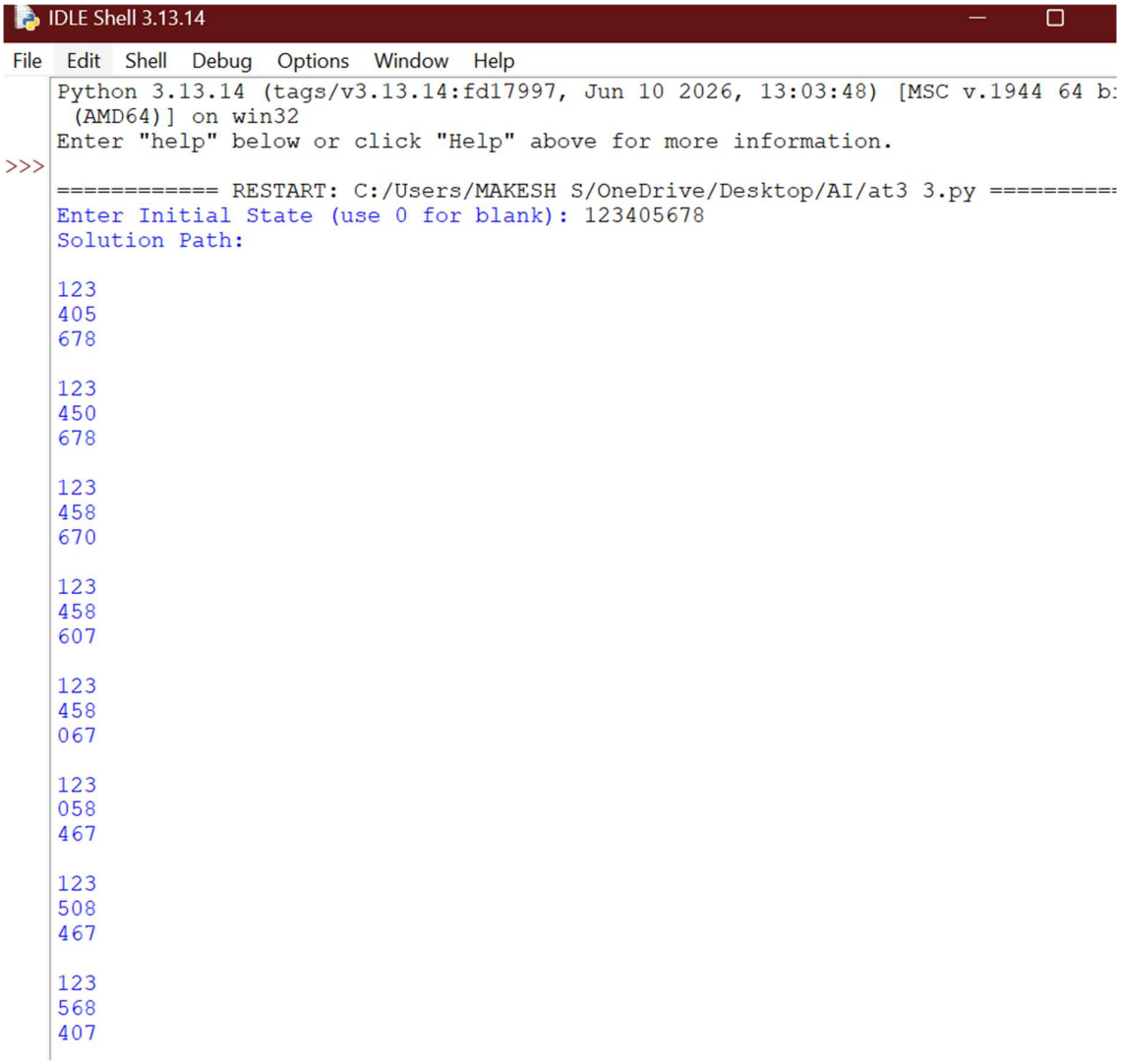
---

3. 8-Puzzle Problem using BFS

```
from collections import deque
goal = "123456780"
moves = {
    0:[1,3],
    1:[0,2,4],
    2:[1,5],
    3:[0,4,6],
    4:[1,3,5,7],
    5:[2,4,8],
    6:[3,7],
    7:[4,6,8],
    8:[5,7]
}
```

```
def bfs(start):
    queue = deque([(start, [])])
    visited = set()
    while queue:
        state, path = queue.popleft()
        if state == goal:
            return path + [state]
        if state in visited:
            continue
        visited.add(state)
        blank = state.index("0")
        for move in moves[blank]:
            temp = list(state)
            temp[blank], temp[move] = temp[move], temp[blank]
            queue.append(("".join(temp), path + [state]))
start = input("Enter Initial State (use 0 for blank): ")
result = bfs(start)
if result:
    print("Solution Path:\n")
    for s in result:
        print(s[:3])
        print(s[3:6])
        print(s[6:])
        print()
else:
    print("No Solution")
```

Output


IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 b
(AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/MAKESH S/OneDrive/Desktop/AI/at3 3.py ==========
Enter Initial State (use 0 for blank): 123405678
Solution Path:
123
405
678
123
450
678
123
458
670
123
458
607
123
458
067
123
058
467
123
508
467
123
568
407